Evaluación Práctica 03

TI3041 - Programación Back End

Unidad de Aprendizaje: 3.API restful con integración a base de datos
Aprendizaje Esperado: 3.1 Codifica una API RESTful para conectarse con otras aplicaciones, utilizando autenticación JWT y herramientas de inteligencia artificial como apoyo al desarrollo.

I. Presentación.

En esta actividad aprenderás a diseñar, estructurar y construir una API RESTful profesional utilizando Django y Django REST Framework. Asimismo, desarrollarás habilidades clave para gestionar peticiones HTTP, controlar excepciones, asegurar el correcto manejo de respuestas e integrar buenas prácticas de documentación, consolidando los fundamentos técnicos necesarios para crear servicios backend escalables y listos para producción..

II. Criterios de Evaluación.

3.1.1 Configura Django Rest framework, según la documentación oficial.
3.1.2 Codifica instrucciones de autenticación, de acuerdo con los requerimientos y considerando recomendaciones de seguridad generadas por herramientas de inteligencia artificial.
3.1.3 Codifica instrucciones que generen resultados en formato JSON, de acuerdo con los requerimientos.
3.1.4 Codifica una API funcional con características RESTful, considerando recomendaciones generadas por herramientas de inteligencia artificial.

III. Evaluación.

- Evaluación: Sumativa con Rúbrica
- Ponderación: 30%
- Modalidad: Actividad Individual
- Plazo: 03/11/2026

IV. Instrucciones.

- Revisar el instrumento de escala de apreciación que se encuentra al final del documento.
- La actividad es individual.
- Utilice Visual Studio Code para construir un proyecto que publique los Endpoints que permitan por un lado, publicar los permisos sobre una aplicación de un usuario y por otro lado, la creación de los datos.
- Utilice Visual Studio Code para construir un proyecto que consuma las API construidas en el otro proyecto.
- El lenguaje de programación es Python usando el framework Django.
- Los datos del proyecto de las APIs se almacenarán en sqlite.

V. Detalle Evaluación

La arquitectura del software ha cambiado, ahora parte de la funcionalidad estará en una API, por lo tanto se necesita construir dos proyectos:



Proyecto 1.

Debe ser un proyecto que publicará Endpoints (servicios).

- ☐ autentificar: recibe el nombre de usuario y la password y retorna un json con los servicios
- ☐ agregarusuario: crea un nuevo usuario
- ☐ actualizar: cambia la contraseña
- ☐ addpermiso: crea un permiso al sistema
- ☐ asociapermiso: asocia un permiso a un usuario

Este proyecto debe guardar los datos en un archivo sqlite, los datos pueden ser creados directamente en la interfaz de los endpoints, no desde el siguiente proyecto

Proyecto 2.

Un sistema de información con un login, y una página principal.
El objetivo es verificar si el usuario tiene acceso usando la API
El menú principal debe configurarse (las opciones se deben ver o no) según los permisos que tiene el usuario.
No ocupe formularios automáticos ni cree interfaces con el admin.
En esta evaluación no se necesita base de datos para este proyecto.

IV. RECURSOS DE APOYO Y PLAZO DE ENTREGA

Material disponible en el ambiente de aprendizaje y Documentación oficial: https://www.django-rest-framework.org/tutorial/quickstart/

El plazo de entrega del trabajo es el 02/11 dentro del horario de clases.

NO se recibirán archivos por correo o fuera de plazo, por lo tanto tome todas las precauciones para enviar correctamente el resultado de la evaluación.

V. PAUTA DE EVALUACIÓN

Criterio / Descriptor	Excelente (4 pts)	Bueno (3 pts)	Regular (2 pts)	Deficiente (1 pt)	Insuficiente (0 pts)
1. Proyecto 1: Endpoint autentificar	El endpoint recibe usuario y contraseña correctamente, valida las credenciales y responde un JSON completo y bien estructurado con los servicios/permisos correspondientes.	El endpoint autentica y retorna el JSON con permisos, pero presenta inconsistencias menores en la estructura del JSON o en la validación.	El endpoint autentica al usuario, pero no retorna la estructura JSON esperada o falla al procesar datos inválidos.	El endpoint está implementado parcialmente, con errores al autenticar o al responder.	No implementa el endpoint autentificar o no funciona en absoluto.
2. Proyecto 1: Endpoints agregarusuario y actualizar	Crea correctamente nuevos usuarios e implementa la actualización/cambio de contraseña de forma segura y funcional.	Implementa la creación de usuario y cambio de contraseña, pero presenta detalles menores de validación o manejo de errores.	Solo funciona uno de los dos endpoints (agregarusuario o actualizar) o presentan fallas de ejecución.	Los endpoints existen pero generan errores continuos o no guardan los cambios correctamente.	No implementa los endpoints de gestión de usuario.
3. Proyecto 1: Endpoints addpermiso y asociapermiso	Permite la creación de permisos del sistema y su correcta asociación a los usuarios de manera consistente.	Crea permisos y los asocia a usuarios, pero carece de verificaciones ante duplicados o errores de asociación.	Permite crear permisos o asociarlos, pero no logra completar ambos flujos adecuadamente.	Los endpoints de permisos están incompletos o presentan errores severos al relacionar datos.	No implementa los endpoints addpermiso ni asociapermiso.
4. Proyecto 1: Persistencia en SQLite	La base de datos SQLite almacena correctamente todas las entidades y relaciones requeridas (usuarios, permisos, asociaciones).	Los datos se guardan en SQLite, pero la estructura del modelo presenta pequeñas deficiencias.	Almacena datos en SQLite pero con inconsistencias en las relaciones o pérdida puntual de datos.	Se configuran modelos/SQLite, pero los datos no persisten de manera confiable.	No utiliza SQLite o no guarda la información solicitada.
5. Proyecto 2: Login y Consumo de API	El sistema posee pantalla de login que valida las credenciales consumiendo exitosamente la API del Proyecto 1.	El login consume la API y valida el acceso, pero presenta demoras o falta de mensajes de error al usuario.	El login intenta consumir la API, pero requiere intervención manual o falla esporádicamente en la comunicación.	El login no se conecta adecuadamente a la API o realiza la validación de forma local sin consumir el servicio.	No implementa la pantalla de login ni el consumo de la API de acceso.

6. Proyecto 2: Configuración Dinámica del Menú	El menú principal oculta o muestra dinámicamente sus opciones según los permisos retornados por la API para el usuario autenticado.	El menú responde a los permisos de la API, pero algunas opciones no se filtran o renderizan correctamente.	El menú se configura parcialmente con los datos de la API, mostrando inconsistencias entre usuarios.	La estructura del menú está creada pero no responde dinámicamente a los permisos recibidos.	El menú principal es estático o no considera los permisos del usuario.
7. Proyecto 2: Cumplimiento de Restricciones Técnicas	Desarrolla la interfaz sin formularios automáticos ni admin de Django, y prescinde completamente de BD en este proyecto.	Cumple las restricciones, pero incluye archivos o configuraciones innecesarias de base de datos sin utilizarlas.	Utiliza elementos automáticos menores o requiere librerías no solicitadas para la interfaz.	Utiliza el admin de Django, formularios automáticos o depende de una base de datos local en el Proyecto 2.	Incumple totalmente las restricciones de desarrollo indicadas.